

Project Executable Files & Execution Guide

Date	
Team ID	
Project Name	OptiCrop: Smart Agricultural Production Optimization Engine
Maximum Marks	3 Marks

Project Executable Files & Execution Guide:

To successfully evaluate our project, evaluators require access to our source code, live deployments, and clear instructions on how to build and run our application locally. Completed the sections below to document our project repositories and execution steps.

Step 1: Project Links & Access

Resource Type	URL / Access Link
Source Code Repository (e.g., GitHub, GitLab)	https://github.com/Kanthisriamarthala/OptiCrop

Live Deployed Application (e.g., Vercel, AWS, Heroku)	https://opti-crop-ks4.vercel.app/
Demo Video Link (Optional but recommended)	https://drive.google.com/file/d/1oMuWCZriScKutjqzOasb4vKEluSsBVIP/view?usp=drive_link

Step 2: Key Executable Files & Folder Structure

Briefly explained the primary directories and where the core execution files are located within our repository.

S.No	Folder / File Name	Description & Purpose
1	/backend/app.py	Main entry point for the Flask REST API server. Defines all 8 API endpoints (/api/health, /api/predict, /api/suitability, /api/dashboard, /api/metrics, /api/crop-info, /api/images, /api/crops). Initializes Flask-CORS and loads ML model.

2	/backend/model/predict.py	Core prediction engine. Contains predict_crop() and assess_suitability() functions. Loads trained Random Forest model and label encoder to predict based on 7 input features.
3	/backend/model/train_model.py	Model training pipeline. Trains and evaluates 6 ML classifiers (Random Forest, SVM, Decision Tree, Logistic Regression, KNN, Naive Bayes) with cross-validation. Also runs K-Means clustering with PCA. Saves all artifacts (.pkl, metrics.json).
4	/backend/utills/visualization.py	Generates all 10 chart images (crop distribution, NPK, temperature, humidity, rainfall, pH, correlation heatmap, boxplots, pairplot, K-Means clusters). Called on server start.
5	/backend/utills/preprocessing.py	Loads and preprocesses Crop_recommendation.csv. Returns clean DataFrame for training and dashboard statistics computation.
6	/backend/config.py	Central configuration. Contains CROP_INFO dictionary (22 crops with description, ideal temp/rainfall/pH, tips), file paths, and server settings.
7	/backend/requirements.txt	Python dependency list (Flask, Flask-CORS, scikit-learn, pandas, numpy, matplotlib, seaborn, scipy, joblib, gunicorn).
8	/frontend/recommend.html	Crop recommendation UI. Form collects 7 parameters, sends POST to /api/predict, renders result card with crop name, confidence %, and detailed crop info.

9	/frontend/dashboard.html	Research dashboard. Fetches /api/dashboard, renders model comparison table, Chart.js bar chart, dataset statistics, correlation insights, K-Means card, 10 chart images.
10	/frontend/js/script.js	Client-side JavaScript. Defines API_BASE global, form validation, contact form handler, tooltips, alert autodismiss, smooth scroll.

Step	Action / Terminal Command	Expected Outcome / Notes
------	---------------------------	--------------------------

1	git clone https://github.com/[your-username]/OptiCrop.git	Downloads the project to the local machine
---	---	--

Step 3: Local Setup & Execution Instructions

Provided a step-by-step guide to installing dependencies and launching the application on a local machine.

3	cd backend python app.py	Launches the Flask backend API server on http://localhost:5000 Auto-generates 10 chart images on first run.
4	Verify backend is running: curl http://localhost:5000/api/health	Expected response: { "status": "ok", "model_ready": true }
5	Open frontend/index.html in browser (or use VS Code Live Server on frontend/ folder)	Opens the OptiCrop homepage. Navigate to Recommend, Suitability, or Dashboard.
6	Open frontend/recommend.html Fill in N, P, K, temp, humidity, pH, rainfall Click Predict Crop	Frontend calls http://localhost:5000/api/predict Displays predicted crop, confidence %, description, ideal conditions, tips.
7	Open frontend/suitability.html Enter parameters and click Analyze Suitability	Frontend calls http://localhost:5000/api/suitability Shows suitability status, productivity level, environmental badges, recommendations.

8	Open frontend/dashboard.html	Automatically fetches /api/dashboard and renders: - Summary cards & model comparison table - Chart.js bar chart (accuracy/precision/recall/F1) - Dataset statistics & correlation insights - K-Means cluster analysis card - 10 chart images (via API or local fallback)
9	(Optional) Test prediction via curl: curl -X POST http://localhost:5000/api/predict ^ -H "Content-Type: application/json" ^ -d "{"nitrogen":90,"phosphorous":40,"potassium":40,"temperature":20,"humidity":80,"ph":7,"rainfall":100}"	Expected response: <pre>{"crop":"rice","confidence":0.28,"crop_info":{"..."}}</pre>
10	Deploy frontend to Vercel: cd frontend && vercel Deploy backend to Render: Push backend/ folder to GitHub Set start command: gunicorn app:app --bind 0.0.0.0:\$PORT Set FRONTEND_URL env var to Vercel URL	Frontend: https://opti-cropks4.vercel.app/ Backend: Render (auto HTTPS, auto-restart)